JAVA FUNDAMENTALS

100 Questions with Simple Answers

Beginner-friendly revision notes covering Java basics, variables, arrays, strings, OOP, access modifiers, packages, and exception handling.

Prepared for study and exam revision

A. Java Fundamentals

1. What is Java?

Answer: Java is a high-level, class-based programming language used to develop desktop, web, mobile, and enterprise applications.

2. What are the main features of Java?

Answer: Java is simple, object-oriented, platform-independent, secure, robust, portable, multithreaded, and supports automatic memory management.

3. Why is Java called platform independent?

Answer: Java source code is compiled into bytecode. The bytecode can run on any operating system that has a compatible Java Virtual Machine (JVM). This is called “Write Once, Run Anywhere.”

4. What is the difference between source code, bytecode, and machine code?

Answer: Source code is the Java program written by a programmer. Bytecode is the intermediate code produced by the Java compiler. Machine code is processor-specific instructions executed by the computer.

5. What is the JVM?

Answer: The Java Virtual Machine executes Java bytecode. It also manages memory and performs tasks such as garbage collection.

6. What is the JDK?

Answer: The Java Development Kit is a collection of tools used to develop Java programs. It includes the compiler, launcher, development tools, and the runtime components needed to run Java programs.

7. What is the JRE?

Answer: The Java Runtime Environment traditionally means the JVM and libraries required to run Java applications. In modern Java distributions, a separate JRE download is not always provided.

8. What is the difference between JDK, JRE, and JVM?

Answer: JDK is used to develop and run Java programs; the JRE concept refers to the runtime libraries and environment; the JVM executes bytecode. The JVM is part of the runtime, and development kits include the tools needed to compile programs.

9. What is bytecode?

Answer: Bytecode is the platform-neutral intermediate instruction code generated by the Java compiler and stored in .class files. The JVM executes it.

10. Why is Java considered an object-oriented programming language?

Answer: Java organizes most program logic using classes and objects and supports encapsulation, inheritance, polymorphism, and abstraction. Primitive values are not objects.

11. What is the role of the main() method in Java?

Answer: The main() method is the traditional entry point from which the JVM starts a standard Java application.

12. Why is main() declared as public static void?

Answer: public lets the launcher access it; static allows it to be called without creating an object; void means it returns no value. String[] args receives command-line arguments.

13. Can we execute a Java program without a main() method?

Answer: A conventional standalone Java application needs a suitable main entry point. Some environments, such as certain frameworks, applets in older Java, or newer Java launch modes, may use different launch mechanisms.

14. What is a Java class?

Answer: A class is a blueprint that defines the data (fields) and behavior (methods) that its objects can have.

15. What is an object?

Answer: An object is an instance of a class. It has state, represented by fields, and behavior, represented by methods.

16. What is the difference between a class and an object?

Answer: A class is the blueprint; an object is a particular instance created from that blueprint. For example, Car is a class and myCar can be an object of Car.

17. What is a reference variable?

Answer: A reference variable stores a reference to an object, or null if it refers to no object. It does not directly store the object's entire contents.

18. What is the difference between primitive data types and reference data types?

Answer: Primitive variables store values such as numbers or true/false. Reference variables refer to objects or arrays and can be null.

19. What are the eight primitive data types in Java?

Answer: byte, short, int, long, float, double, char, and boolean.

20. What is type casting?

Answer: Type casting converts a value from one data type to another. Widening conversions, such as int to long, are generally automatic; narrowing conversions, such as double to int, require an explicit cast and may lose information.

B. Variables, Operators and Control Statements

21. What is a local variable?

Answer: A local variable is declared inside a method, constructor, or block. It must be initialized before it is used and is available only within its scope.

22. What is an instance variable?

Answer: An instance variable is declared in a class but outside methods and constructors, without static. Each object has its own copy.

23. What is a static variable?

Answer: A static variable belongs to the class rather than to individual objects. One shared copy is associated with the class.

24. What is the difference between == and =?

Answer: The = operator assigns a value. The == operator compares primitive values or checks whether two reference variables refer to the same object.

25. What is the difference between == and .equals()?

Answer: For primitives, == compares values. For references, == compares object identity. equals() compares objects according to the class's implementation; String.equals() compares text content.

26. What are relational operators in Java?

Answer: Relational operators compare values: <, >, <=, >=, ==, and !=. They produce a boolean result.

27. What are logical operators?

Answer: Logical operators combine or negate boolean conditions. The main operators are && (conditional AND), || (conditional OR), and ! (NOT).

28. What is the difference between && and &?

Answer: For boolean expressions, && stops evaluating if the first condition is false, while & evaluates both sides. & can also be used as a bitwise AND operator on integer values.

29. What is the ternary operator?

Answer: The ternary operator ?: is a short form of an if-else expression. Example: int max = (a > b) ? a : b;

30. What is the difference between break and continue?

Answer: break exits the nearest loop or switch. continue skips the rest of the current loop iteration and proceeds to the next iteration.

31. What is the difference between while and do-while?

Answer: while checks the condition before executing the body, so it may run zero times. do-while checks after the body, so it runs at least once.

32. When would you use a for loop?

Answer: Use a for loop when repeating an operation a known number of times or when initialization, condition, and update fit neatly together.

33. What is a nested loop?

Answer: A nested loop is a loop placed inside another loop. For each outer-loop iteration, the inner loop can run through its iterations.

34. What is a switch statement?

Answer: A switch selects and executes a branch based on the value of an expression, often making multiple alternatives clearer than a long if-else chain.

35. Can a switch statement work with a String?

Answer: Yes. Java supports String values in switch statements, starting with Java 7.

36. What happens if break is omitted from a switch case?

Answer: In traditional colon-style switch cases, execution can continue into the following case labels (fall-through) until a break or the end of the switch. Arrow-style cases do not fall through.

C. Arrays and Strings

37. What is an array?

Answer: An array stores a fixed number of values of the same element type, accessed using indexes starting at 0.

38. Why are arrays called fixed-size data structures?

Answer: Once an array is created, its length cannot be changed. To hold a different number of elements, create another array or use a resizable collection such as ArrayList.

39. How do you declare and initialize an array in Java?

Answer: Example: int[] marks = {80, 90, 75}; Or create an array with a chosen length: int[] marks = new int[3];

40. What is the default value of elements in an integer array?

Answer: Every int element defaults to 0. Other primitive arrays receive their type's default value; reference-type array elements default to null.

41. How do you find the length of an array?

Answer: Use the length field, without parentheses. Example: marks.length.

42. What happens when you access an invalid array index?

Answer: Java throws ArrayIndexOutOfBoundsException at runtime.

43. What is a two-dimensional array?

Answer: A two-dimensional array is an array whose elements are themselves arrays. It is commonly used to represent rows and columns, for example int[][] matrix = new int[2][3].

44. What is the difference between an array and an ArrayList?

Answer: An array has a fixed length and can store primitives directly. ArrayList can grow or shrink and stores objects (primitive values are handled through wrapper classes such as Integer).

45. What is a String in Java?

Answer: String is a class representing a sequence of characters. String objects are immutable.

46. Why is String immutable?

Answer: A String's contents cannot be changed after creation. Immutability supports safe sharing, predictable behavior, and reliable use in structures such as the string pool and hash-based collections.

47. What is the difference between String, StringBuilder, and StringBuffer?

Answer: String is immutable. StringBuilder is mutable and usually preferred for repeated changes in single-threaded code. StringBuffer is mutable and its methods are synchronized for thread safety.

48. What is the difference between equals() and equalsIgnoreCase()?

Answer: equals() compares String contents with case sensitivity. equalsIgnoreCase() compares contents without distinguishing uppercase and lowercase letters.

49. What is the use of length() in a String?

Answer: length() returns the number of UTF-16 code units in the String. Example: "Java".length() returns 4.

50. What is the use of substring()?

Answer: substring() extracts part of a String. The starting index is included and the ending index is excluded. Example: "Java".substring(1, 3) returns "av".

51. What is the use of charAt()?

Answer: charAt(index) returns the UTF-16 char at the given zero-based index. An invalid index causes StringIndexOutOfBoundsException.

52. What is the difference between trim() and strip()?

Answer: trim() removes leading and trailing characters whose code points are at most U+0020. strip(), available from Java 11, uses Unicode whitespace rules.

53. What do toUpperCase() and toLowerCase() do?

Answer: They return a String converted to uppercase or lowercase. They do not change the original String because Strings are immutable.

54. What is the use of contains() in String?

Answer: contains() checks whether a String contains a specified character sequence and returns true or false.

55. What is the difference between concat() and the + operator for Strings?

Answer: Both can combine Strings. + is more convenient and can combine values of different types through string conversion. concat() takes a String argument. Neither changes the original String.

D. OOP Concepts

56. What are the four main principles of OOP?

Answer: Encapsulation, inheritance, polymorphism, and abstraction.

57. What is encapsulation?

Answer: Encapsulation combines data and the methods that operate on it in a class and controls access to the internal state.

58. How is encapsulation achieved in Java?

Answer: Declare fields private and provide suitable public methods, such as getters and setters, to read or update them with validation when needed.

59. What is inheritance?

Answer: Inheritance allows a class to acquire accessible fields and methods from another class. In Java, a class uses extends to inherit from a class.

60. Why is inheritance used?

Answer: Inheritance supports code reuse, represents an “is-a” relationship, and allows subclasses to specialize or override inherited behavior.

61. What is the difference between extends and implements?

Answer: extends is used to inherit from a class or to extend an interface. implements is used by a class to promise that it provides the methods required by an interface.

62. What is polymorphism?

Answer: Polymorphism means one interface or parent-type reference can represent different object types, allowing the appropriate behavior to be selected.

63. What is compile-time polymorphism?

Answer: Compile-time polymorphism is commonly achieved through method overloading. The compiler selects a matching method based on the argument list and types.

64. What is runtime polymorphism?

Answer: Runtime polymorphism occurs when an overridden instance method is selected according to the actual object's class during execution.

65. What is method overloading?

Answer: Method overloading means defining methods with the same name but different parameter lists in the same class or inheritance context.

66. What is method overriding?

Answer: Method overriding occurs when a subclass provides a compatible implementation of an inherited instance method, using the same signature and a compatible return type.

67. What is the difference between method overloading and overriding?

Answer: Overloading changes the parameter list and is resolved at compile time. Overriding redefines inherited instance-method behavior and is dispatched at runtime.

68. Can we overload a method by changing only its return type?

Answer: No. Changing only the return type does not create a distinct overload; the parameter list must differ.

69. What is the super keyword?

Answer: super refers to the immediate superclass context. It can call a superclass constructor with super(...) or access superclass members hidden or overridden in the subclass.

70. What is the this keyword?

Answer: this refers to the current object. It can distinguish fields from parameters, call another constructor using this(...), or pass the current object as an argument.

71. What is a constructor?

Answer: A constructor initializes a new object. It has the same name as the class and has no return type.

72. What is the difference between a constructor and a method?

Answer: A constructor initializes an object and is invoked during object creation. A method performs an operation, has a return type (possibly void), and is called explicitly or through program behavior.

73. Can a constructor be overloaded?

Answer: Yes. A class can have multiple constructors with different parameter lists.

74. Can a constructor be inherited?

Answer: No. Constructors are not inherited, although a subclass constructor can call a superclass constructor.

75. What is an abstract class?

Answer: An abstract class is declared with abstract and cannot be instantiated directly. It can contain abstract methods and implemented methods, fields, and constructors.

76. What is an interface?

Answer: An interface defines a contract that implementing classes agree to follow. It can declare abstract methods, constants, and also default, static, and private methods where supported by the Java version.

77. Why do we use interfaces?

Answer: Interfaces define common behavior for different classes, support loose coupling, and allow a class to implement multiple interface types.

78. What is the difference between an abstract class and an interface?

Answer: An abstract class can hold instance state and constructors and can be extended by only one class. A class can implement multiple interfaces; interfaces mainly define contracts and do not have ordinary per-object instance fields.

79. Can a class implement multiple interfaces?

Answer: Yes. Example: class Car implements Accelerate, Brake { ... }

80. Can an interface extend another interface?

Answer: Yes. An interface can extend one or more interfaces using extends.

E. Access Modifiers, Packages & Core Concepts

81. What are the access modifiers in Java?

Answer: The four access levels are private, package-private (default, when no modifier is written), protected, and public.

82. What is the difference between private, default, protected, and public?

Answer: private: accessible only within the declaring class. Default/package-private: accessible within the same package. protected: accessible within the same package and, under inheritance rules, from subclasses in other packages. public: accessible wherever the type/member is otherwise accessible.

83. Why are data members generally declared private?

Answer: Private fields protect an object's internal state and allow controlled access through methods, making validation and future changes easier.

84. What is a package?

Answer: A package is a namespace used to group related Java types, such as classes and interfaces.

85. Why are packages used in Java?

Answer: Packages organize code, help avoid naming conflicts, support access control, and make large projects easier to maintain.

86. What is the difference between import and package?

Answer: package declares which package a source file's type belongs to. import lets source code refer to types from other packages without writing their fully qualified names each time.

87. What is the purpose of the static keyword?

Answer: static makes a field or method belong to the class rather than to a particular object. Static methods can be called using the class name and cannot directly use this.

88. What is the purpose of the final keyword?

Answer: final prevents reassignment of a variable, overriding of a method, or inheritance from a class, depending on where it is used.

89. Can a final variable be modified?

Answer: A final variable cannot be assigned a new value after it has been initialized. If it refers to a mutable object, the object's contents may still be changed.

90. Can a final method be overridden?

Answer: No. A final method cannot be overridden by a subclass.

F. Exception Handling

91. What is an exception?

Answer: An exception is an event represented by an object that interrupts the normal flow of a program, such as dividing an integer by zero or opening a missing file.

92. What is exception handling?

Answer: Exception handling is the process of detecting and responding to exceptional conditions using constructs such as try, catch, finally, throw, and throws.

93. What is the difference between an error and an exception?

Answer: Both are subclasses of Throwable. Exceptions are conditions applications may often handle. Errors usually indicate serious JVM or system problems and are generally not intended to be handled by ordinary application code.

94. What is the difference between checked and unchecked exceptions?

Answer: Checked exceptions must be caught or declared with throws, subject to Java's compile-time rules. Unchecked exceptions are RuntimeException subclasses (and Error subclasses) and do not have that requirement.

95. What is the purpose of try and catch?

Answer: The try block contains code that may throw an exception. A matching catch block handles the exception so the program can respond appropriately.

96. What is the purpose of the finally block?

Answer: finally is used for cleanup code that should normally run whether or not an exception occurs. For resources, try-with-resources is often safer. finally may not run if the JVM terminates abruptly.

97. What is the difference between throw and throws?

Answer: throw explicitly throws a particular exception object, for example throw new IllegalArgumentException(). throws appears in a method declaration to list exceptions that may propagate to the caller.

98. Can we create our own exception class?

Answer: Yes. Define a class that extends Exception for a checked custom exception or RuntimeException for an unchecked one.

99. What happens if an exception is not handled?

Answer: If no matching handler is found, the exception propagates up the call stack. If it remains uncaught on the thread, that thread terminates and the runtime typically prints a stack trace.

100. Why should exceptions be handled instead of allowing the program to terminate unexpectedly?

Answer: Handling exceptions allows a program to show useful messages, recover when possible, release resources, and fail in a controlled way. Exceptions should be handled meaningfully rather than hidden.